

Research Note: Low-Level Design (LLD) Practice & Evaluation Platform

Author: Engineering Assignment Candidate | Scope: Learner Problem, Competitive Analysis, Gaps & Direction | Length: 2 Pages

1. Executive Summary & Problem Space

Low-Level Design (LLD) — also known as Object-Oriented Design (OOD) or Machine Coding — evaluates an engineer's ability to model real-world business domains through clean classes, interfaces, design patterns, and SOLID principles. While candidates have extensive theoretical resources, **practicing and evaluating LLD remains structurally broken**.

In Data Structures & Algorithms (DSA), evaluation is binary and deterministic: *Input* → *Output* → *Pass/Fail (Time & Space Complexity)*. In LLD, learners face an ambiguous, open-ended problem space:

- **No Single 'Correct' Answer:** A Parking Lot or Elevator can legitimately be built with Strategy, State, Observer, or Factory patterns. Learners cannot discern whether their abstractions are elegant or over-engineered.
- **Working Code vs. Good Design:** A learner can write 200 lines of spaghetti code that executes flawlessly yet completely violates Single Responsibility (SRP) and Open/Closed (OCP) principles.
- **Absence of Scalable, Actionable Feedback:** Without a senior staff engineer conducting an interactive design review, learners cannot gauge coupling, extensibility, or thread-safety invariants.

2. Analysis of Existing Approaches & Market Tools

Approach / Tool	How It Operates	Key Strengths	Critical Gaps in LLD Practice
LeetCode / HackerRank	Automated unit test assertion against I/O.	Instant execution, automated benchmarks, fast feedback.	Zero architectural critique. Encourages anti-patterns (God methods, global static state) just to pass tests.
Generic LLMs (ChatGPT / Claude)	Unstructured chat prompts ('Review my design').	Conversational, flexible, accessible on demand.	Non-deterministic rubric & praise bias. Suffers from hallucinations, lacks AST contract verification, and has no attempt history.
Static Books & Courses (Alex Xu, Head First, Grokking)	Case-study walkthroughs of canonical systems.	Canonical class diagrams, clean pattern breakdowns.	Passive reading with zero interactive practice. Creates 'illusion of competence' without hands-on authoring.
Human Peer Code Reviews	Mock interviews (Pramp, GitHub PR reviews).	High contextual nuance and conversational trade-offs.	Expensive, unscalable, and slow. Turnaround takes days; quality varies wildly depending on reviewer bias.

3. Four Structural Gaps Identified

- **Gap 1: The Multi-Modal Submission Void:** Real LLD consists of three unified artifacts: (1) Core class/interface contracts, (2) Explicit design rationale & assumptions, and (3) Collaborative usage walkthrough. Existing tools only accept a single code box, discarding architectural intent.
- **Gap 2: Binary Grading vs. Multi-Dimensional Rubrics:** LLD cannot be scored binary pass/fail. Meaningful evaluation requires orthogonal scoring: Domain Requirements, SOLID Adherence, Extensibility/Patterns, and Edge-Case Invariants.
- **Gap 3: Absence of 'What-If' Stress Testing:** Senior interviewers test adaptability: *'What if we add surge pricing or EV charging spots?'* Existing automated tools never stress-test abstractions against downstream requirement changes.
- **Gap 4: Broken Iterative Progression:** Mastery requires a loop: submitting, reviewing critique, refactoring, and observing score improvement. Existing tools lack versioned attempt diffing.

4. Product Direction & Core Tenets for the LLD Practice Platform

To solve these gaps, we designed the **LLD Practice Platform** around a focused, repeatable learner loop: *Choose Problem* → *Think & Design* → *Submit* → *Explainable Feedback* → *Review & Refine*.

Tenet 1: Dual-Track Hybrid Evaluation Engine

Rather than choosing between pure rigid unit tests and hallucination-prone LLM prompts, we architected a **two-tiered evaluation pipeline**:

- **Deterministic AST & Structural Rules (Fast, 100% Consistent)**: Analyzes class ASTs to verify expected domain entity coverage, detect God Classes (>6 methods), check interface abstraction, flag concrete instantiations violating Dependency Inversion (DIP), and detect switch-on-type anti-patterns.
- **Semantic Reasoning (Qualitative & Socratic)**: Leverages LLM reasoning to evaluate trade-off notes, pattern elegance, and generate dynamic 'What-If' scenarios. If external AI times out (>7s) or is unconfigured, a deterministic fallback heuristic engine guarantees instant, high-quality feedback without crashing.

Tenet 2: Multi-Modal Tri-Part Practice Studio

Learners author their designs across three synchronized panes: (1) **Domain Code Editor** (TypeScript/OOP class and interface definitions), (2) **Design Rationale Notes** (explicit assumptions on concurrency and trade-offs), and (3) **Real-Time Live UML Diagram** (dynamically generated in-browser via Mermaid.js from the code AST to visualize relationships as they type).

Tenet 3: Rubric-Grounded, Explainable Feedback

Every evaluation produces a transparent scorecard across 4 universal pillars (0–100 weighted score):

- **Functional Requirements (25%)**: Entity completeness and lifecycle abstraction.
- **SOLID Principles (25%)**: Granular checklist with pass/fail badges for SRP, OCP, LSP, ISP, and DIP.
- **Extensibility & Patterns (25%)**: Detection of behavioral patterns (Strategy, State, Factory) vs. brittle coupling.
- **Edge Cases & Invariants (25%)**: Capacity defense, error handling, and thread-safety awareness.

Critiques avoid stylistic naming bias and provide: (1) The Architectural Consequence, (2) Concrete Refactoring Nudge, and (3) Dynamic 'What-If' Stress Challenge.

Tenet 4: Versioned Attempt Progression & Mastery Tracking

Every problem attempt is version-tracked (v1, v2, v3). Learners compare rubric score trajectories over time and can restore historical code with one click, turning vague self-doubt into measurable architectural growth.

5. Summary & Educational Impact

By replacing opaque binary tests with a transparent, rubric-driven hybrid evaluation loop, the LLD Practice Platform bridges the gap between passive textbook study and real-world system design interviews. It empowers software engineers to practice repeatedly, receive explainable senior-level architectural guidance, and master low-level design with confidence.